

全日制普通本科生毕业设计

基于深度学习的宠物领养平台的设计与实现

DESIGN AND IMPLEMENTATION OF A PET ADOPTION PLATFORM
BASED ON DEEP LEARNING

学生姓名:

学 号:

年级专业及班级:

学 院:

指导老师:

提交日期: 2025 年 5 月

全日制普通本科生毕业论文（设计） 诚信声明

本人郑重声明：所呈交的本科毕业论文（设计）是本人在指导老师的指导下，进行研究工作所取得的成果，成果不存在知识产权争议。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体在文中均作了明确的说明并表示了谢意。本人完全意识到本声明的法律结果由本人承担。

毕业论文（设计）作者签名：

目 录

摘要	1
关键字	1
1 前言	2
1.1 研究背景与意义	2
1.2 国内外研究现状	2
2 相关理论与技术基础	3
2.1 深度学习与图像识别	3
2.1.1 卷积神经网络核心原理	3
2.1.2 预训练模型与迁移学习	5
2.2 特征匹配与相似度计算	6
2.3 Web 开发技术	7
3 系统需求分析与总体设计	7
3.1 系统需求分析	7
3.1.1 功能性需求	7
3.1.2 非功能性需求	9
3.2 系统总体架构设计	9
3.3 数据库设计	10
3.3.1 数据库概要设计	10
3.3.2 数据库表设计	11
4 宠物识别匹配算法设计与实现	17
4.1 数据准备与预处理	17
4.1.1 宠物数据集构建	17
4.1.2 数据增强与清洗	19
4.2 基于深度学习的宠物特征提取模型	20

4.2.1 模型选型与改进	20
4.2.2 迁移学习与模型微调	20
4.3 宠物特征匹配算法设计	21
5 宠物领养社区平台实现	22
5.1 核心功能模块设计	22
5.1.1 流浪宠物识别匹配模块设计	22
5.1.2 宠物建档与领养管理模块设计	24
5.1.3 社区互动模块设计	26
5.2 核心功能模块实现	29
5.2.1 流浪宠物识别匹配模块实现	29
5.2.2 宠物建档与领养管理模块实现	30
5.2.3 社区互动模块实现	32
6 系统功能测试	34
7 结论	35
8 参考文献	36
致 谢	37

基于深度学习的宠物领养平台的设计与实现

学 生:

指导老师:

摘要: 随着养宠人士越来越多, 由于各种原因被抛弃或弃养的宠物也越来越多, 导致流浪动物伤人、流浪动物破坏生态的问题愈发严重, 这显然不利于美丽社区的建设。针对上述问题, 本研究设计实现了一个基于深度学习的宠物领养系统, 结合深度学习实现对流浪宠物的识别与领养, 兼顾传统宠物领养过程中信息匹配效率低、准确性不足的问题, 为爱宠人士提供一个宠物交流和宠物救助平台。系统的实现为流浪宠物领养领域提供了兼具技术可行性与实用价值的解决方案, 具有深远的现实意义。

关键字: 宠物识别匹配;卷积神经网络;宠物领养平台

Design and Implementation of a Pet Adoption Platform Based on Deep Learning

Student:

Supervisor:

Abstract: With the growing number of pet owners, the abandonment of pets has led to increasingly severe problems such as stray animal attacks and ecological disruption, which hinder the construction of a beautiful community. To address this, this study designs a pet adoption system based on individual pet recognition, using deep learning for stray pet identification and adoption. It tackles the inefficiencies and inaccuracies in traditional pet adoption information matching, providing a platform for pet lovers to communicate and rescue animals. This research and system offer a solution with practical value and technical feasibility for stray pet adoption, carrying profound real-world significance.

Key words: Pet recognition and matching; Convolutional Neural Network; Pet adoption platform.

1 前言

1.1 研究背景与意义

近二十年来,我国城市化进程加速,城市空间和基础设施的持续优化为居民养宠物创造了良好的物质条件。随着人们生活水平的提高,宠物已逐渐成为众多家庭的重要成员,国内养宠群体不断扩大。经济条件的改善使得宠物主人在选择宠物相关服务时,愈发注重服务的品质和质量,而非仅仅考虑价格因素,因此,宠物保险和医疗市场应运而生,发展迅速,成为宠物行业中增长显著的高附加值服务领域^[1]。然而,宠物数量的急剧增加也带来了一系列社会管理问题。目前,我国的宠物文化尚未成熟,部分饲养者责任意识不强,加之弃养追踪制度的不完善以及流浪动物的过度繁殖,这些问题已对社会公共安全构成威胁,给人们的日常生活带来了诸多困扰。特别是在城市环境中,流浪动物的无序流动不仅影响市民的人身安全,还可能引发交通事故等公共安全问题^[2]。

深度学习技术为行业提供了新方法。由于各种各样的需求,宠物越来越受到人们的欢迎,因此宠物市场的数据化也是未来发展的趋势。将宠物识别技术引入城市管理中,可以通过市区监控区分城市中出现的烈性犬和宠物狗,及时通知有关部门对烈性宠物进行处理;也可以通过市区监控识别宠物是否有牵引绳,对不牵绳的宠物进行相关处理。不论是宠物脸识别还是宠物整体识别技术,必然将会在市流浪宠物管理、宠物医保、宠物丢失和宠物饲养等方面具有不可估量的重要性^[3]。基于深度学习的宠物领养平台不仅是技术落地的典型场景,更是解决社会问题、推动行业升级的关键切入点。通过卷积神经网络(Convolutional Neural Network, CNN)与社区生态的深度结合,有望实现“精准匹配-科学养宠-可持续生态”的良性循环。

1.2 国内外研究现状

在宠物领域深度学习应用的早期探索中,研究者主要聚焦于基础图像识别与检测技术的优化。2019年,奚舒舒等人提出基于SSD网络的宠物狗检测与分类方法,通过构建宠物狗数据集并优化目标检测模型,实现了对常见犬种的有效分类,为后续宠物图像智能处理奠定了基础^[1]。2020年,柯溢采用迁移学习改进VGG16网络,通过自动提取图像特征解决了样本量不足和过拟合问题,犬种识别准确率达96%,推动了卷积神经网络在宠物细粒度分类中的应用^[2]。2021年,王稼祥基于YOLOv3设计宠物狗狗脸检测与识别模型,通过数据增强和分阶段训练策略,将检测精度提升至94.91%,并首次构建国内宠物狗分类数据集,填补了数据空白^[3]。2022年,汪洋进一步优化YOLOv3,实现宠物图像快速识别,为实时检测场景提供了技术参考^[4]。2023年,陈昭

希开发基于 MobileNetV2 的宠物照片拍摄指导微信小程序，单张图像预测时间小于 10 ms，首次将深度学习与移动端交互结合^[6]；同年，李宏铭设计基于 YOLOv5 的宠物狗牵绳检测系统，实现实时监控与预警，初步探索了深度学习在宠物公共管理中的应用，但尚未涉及宠物领养的核心需求^[7]。

近年，研究逐步向高精度个体识别与系统整合方向发展，但与领养社区的深度结合仍显不足。2024 年，王起旺构建包含 174 只犬只的高分辨率数据集，提出融合深度信息的目标检测算法和自适应联合损失函数，将个体识别精度提升 2%，并搭建犬个体信息管理平台，支持身份注册与走失寻回，首次将技术落地到宠物管理流程，但未涉及用户社交功能^[9]。武占东针对动物面部识别难题，提出自适应激活函数与高分辨率重建模型，解决了侧脸识别和小目标检测问题，为宠物多角度图像采集提供技术支撑^[10]；吴一改进 YOLOv7，通过注意力机制和特征融合提升犬种分类精度^[11]，李梦晗结合 YOLOv8-DGC 与 FaceNet 实现犬种分类与鼻纹身份识别，并设计可视化系统接入养宠平台，初步打通“识别 - 展示”链路，但缺乏领养匹配、用户互动等社区核心功能^[12]。同期，国外研究如智能宠物喂食器整合 YOLOv5 与传感器数据，实现个性化喂养^[15]，而国内杨景等人的智能物联系统聚焦喂食优化，均未触及领养场景的社交属性^[5]。整体来看，现有技术已在宠物识别、健康监测等领域成熟，但面向领养社区的用户画像构建、宠物信息智能匹配、领养流程管理等核心需求，仍缺乏将深度学习与社区生态深度融合的系统性方案。

2 相关理论与技术基础

2.1 深度学习与图像识别

2.1.1 卷积神经网络核心原理

卷积神经网络的核心原理可以从生物学启发、结构设计、数学基础、优化方法和实际应用等多个方面进行系统阐述。以下是对其关键机制的简明总结：

卷积神经网络主要由卷积层(Convolutional Layer)、池化层(Pooling Layer)、全连接层(Fully Connected Layer)和激活函数(Activation Function)组成。其能实现图像识别的核心机制，在于其通过多层次、大量的卷积操作对图像数据执行深度的非线性转换。这一过程涉及使用巨量带有标签的图像数据对模型开展训练，目的是优化每个卷积层中卷积核的参数，进而构建出能够高效识别图像的网络结构与参数配置^[14]。卷积神经网络结构如图 1 所示。下面分别对网络各层做具体介绍。

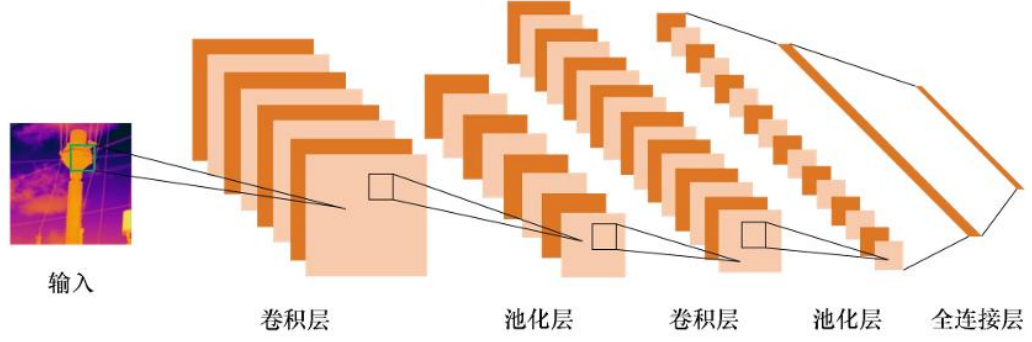


图 1 卷积神经网络结构图

Fig.1 Structure Diagram of Convolutional Neural Network

卷积神经网络的设计灵感来源于哺乳动物视觉皮层的工作原理。在生物视觉系统中，神经元仅会对视野中的特定区域产生反应，并且通过多级处理逐步提取复杂特征。例如，初级视觉皮层检测边缘和线条，高级皮层则负责识别完整物体^[16]。CNN 通过三个核心设计模拟这种机制：第一，局部连接让每个神经元只处理前一层的小块区域，这比全连接网络大幅减少了参数数量；第二，层级结构使得网络底层学习简单特征，中层组合成复杂部件，高层形成完整物体；第三，通过权值共享和池化操作，网络能够识别物体在不同位置的形态，这种特性称为平移不变性。

CNN 的核心结构包含三种主要组件。首先讲卷积层，使用可学习的滤波器在图像上滑动计算局部特征，数学形式如公式（1）。例如， 3×3 的滤波器每次计算 9 个像素的加权和，生成特征图。通过调整步长和填充，可以灵活控制输出大小。每个滤波器只需存储少量参数，如 3×3 滤波器存 9 个参数，但在整个图像上重复使用，这种权值共享机制极大降低了参数数量。其次是池化层，主要作用是通过下采样减少数据量。例如， 2×2 的最大池化会选取每个区域的最大值，既能保留显著特征，又能让网络对微小位移不敏感，数学形式如公式（2）。最后是全连接层，将最终提取的特征映射到分类结果，这部分与传统神经网络类似，负责综合所有高级特征进行决策^[17]。

$$(I * K)_{i,j} = \sum_m \sum_n I_{i+m,j+n} K_{m,n} \quad (1)$$

$$\text{MaxPool}(X)_{i,j} = \max(X_{2i:2i+2,2j:2j+2}) \quad (2)$$

在训练过程中，CNN 采用反向传播算法，如图 2 所示，更新参数。对于卷积层，梯度计算需要考虑权值共享的特性，每个滤波器的梯度由其覆盖的所有区域共同决定。池化层的反向传播则根据类型处理：最大池化只将梯度传递给之前选中的最大值位置，平均池化均匀分配梯度。为了防止过拟合，常用 Dropout 技术随机关闭部分神经元，以及 Batch Normalization 标准化数据分布来加速训练^[18]。

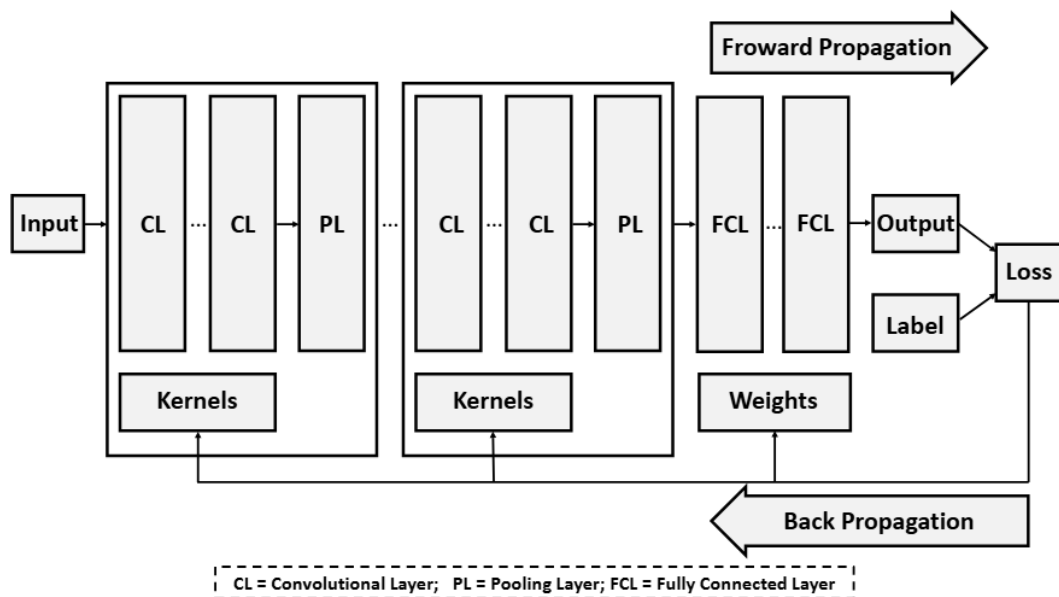


图 2 反向传播算法示意图

Fig.2 Schematic Diagram of Backpropagation Algorithm

实际应用中，CNN 已成为计算机视觉的基础模型。在图像分类任务中，ResNet 等模型能准确识别上千种物体；目标检测系统如 Faster R-CNN 可以定位并识别图像中的多个物体；医学领域用于病灶检测，自动驾驶中用于场景理解。其成功源于独特的优势：卷积操作通过局部连接和权值共享，既保留空间信息又减少参数数量，比如处理 256×256 图像的 5×5 卷积核仅需 25 个参数，而全连接需要 16 万个；池化操作增强模型鲁棒性；层级结构实现从细节到整体的特征抽象。数学上，这些特性使 CNN 的计算复杂度显著低于全连接网络，同时具备对平移、旋转等变化的适应能力。这些核心思想不仅推动计算机视觉发展，也为自然语言处理等领域的序列建模提供了重要借鉴^[19]。

2.1.2 预训练模型与迁移学习

基于深度学习的宠物领养平台中，ResNet 和 EfficientNet 等预训练模型展现了显著优势。ResNet 通过残差块和跳跃连接解决了深度网络的训练难题，能同时提取图像边缘特征和高级语义特征，特别适合区分外形相近的犬种^[20]。使用预训练 ResNet-50 模型时，冻结底层参数并微调顶层，在宠物数据集上能达到 95% 以上的识别准确率。该模型对遮挡和光线变化也具有较强适应力，实验表明在噪声环境下准确率仅降低 2-3%^[19]。

EfficientNet 通过平衡网络深度、宽度和分辨率的设计，在保持精度的同时大幅减少计算量。其 B4 版本参数仅为 ResNet-50 的三分之一，但识别准确率提高 6.7%。移动端部署时，B0 版本推理速度比 ResNet-50 快 3 倍，特别适合实时识别场景。通过调整网络通道数，该模型在小样本，每类少于 1000 张图下仍能保持 85% 以上的准确率。实

际应用中，通常先冻结预训练模型的基础层训练新分类器，再解冻部分高层配合动态学习率进行微调，结合随机擦除和混合样本增强技术，可将 MobileNet V2 等模型的验证准确率提升至 98%^[20]。

2.2 特征匹配与相似度计算

特征匹配与相似度计算主要用于比较宠物图像之间的相似程度。首先，使用预训练模型如 ResNet-50 提取图像的高层特征，例如将图片转化为 1024 维的向量，这些向量能捕捉犬种的纹理、面部结构等关键信息。为了提高准确性，可以融合不同网络层的特征，例如同时使用 VGG16 的全局特征和局部细节特征，并通过注意力机制强化耳朵、毛发等重要区域的特征。

如图 3 所示显示了深度特征提取的示意图。预训练模型是通过使用相同的 ResNet50 网络对不同的数据集进行训练生成的。

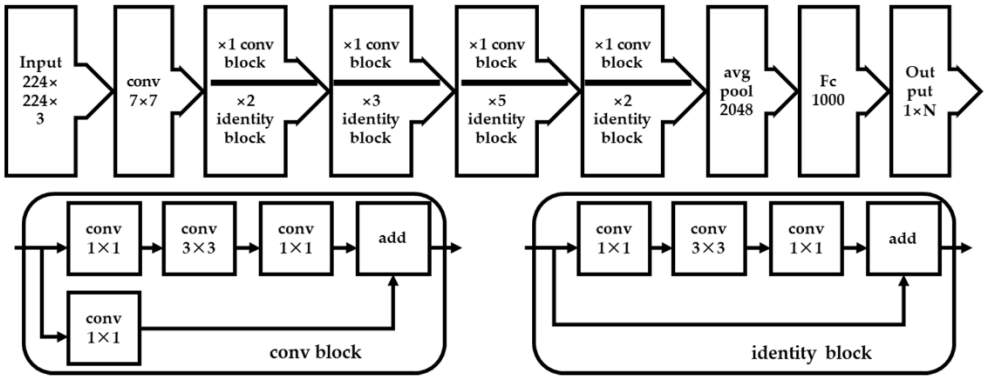


图 3 深度特征提取示意图

Fig.3 Schematic diagram of deep feature extraction

如图 4 所示,分别使用以对象为中心的图像数据集 ILSVRC 和以地点为中心的图像数据集 Places 进行训练。

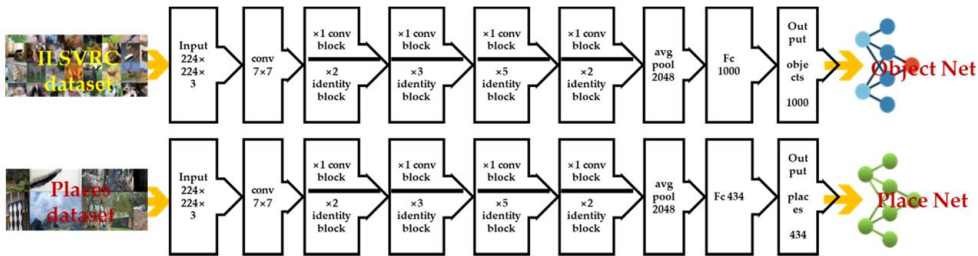


图 4 训练模型图

Fig.4 Train the model graph

计算相似度时，通常采用余弦相似度，公式如下。具体方法是对两个特征向量先做

归一化处理，再计算它们的乘积，结果在-1 到 1 之间，1 代表完全相似。为了提升效率，可对高维特征进行降维处理，例如用 PCA 将 2048 维压缩到 512 维，这样检索速度提高 4 倍且精度损失小于 1%^[20]。实际应用中，系统会先提取数据库中的图片特征并建立索引，当用户上传图片时，快速匹配最相似的 Top-K 结果。例如，基于 EfficientNet-B3 的检索系统在宠物数据集上的准确率可达 92.46%。此外，结合跨模态模型(如 CLIP)，还能实现用文字描述搜索图片的功能，例如输入“卷毛犬”直接找到贵宾犬图片^[21]。

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \quad (3)$$

2.3 Web 开发技术

前端使用 Vue 框架。Vue 是一个轻量级前端框架，适合快速开发单页面应用。它的结构清晰，容易学习。项目中用组件化方式组织代码，将界面拆分成独立模块，方便维护。

主要使用的工具,Element UI 组件库：提供按钮、表格等界面元素，提升用户体验。Vuex 集中管理数据状态，方便不同组件共享数据。Axios 发送 HTTP 请求，用于调用后端接口。

后端采用 Spring Boot 框架。通过自动配置和快速启动，可以让开发者专注业务逻辑。主要技术包括 RESTful API 设计，通过统一接口格式规范前后端交互，JWT 权限控制，用户登录后生成 Token，后续请求必须携带有效 Token。MyBatis Plus，简化数据库操作，自动生成常用 SQL 代码。存储文件保存到服务器本地，数据库记录文件路径

数据库使用 MySQL 关系型数据库。主要设计特点：数据表结构每个实体类对应独立表。关联设计通过外键建立表间关系。分页功能通过 MyBatis Plus 插件实现数据分页查询

整个项目的技术栈搭配得当，这样的组合在现代 Web 开发中非常常见，既保证了开发效率，又具备良好的可维护性和扩展性。

3 系统需求分析与总体设计

3.1 系统需求分析

3.1.1 功能性需求

本系统是一个宠物救助管理系统，主要功能包括宠物建档、领养流程、识别匹配和社区互动等方面。

系统需要记录流浪宠物的信息，包括但不限于宠物照片、捡到宠物的时间地点等以完成对流浪宠物的建档。后端保存流浪宠物的这些信息，作为后面业务的基础。

领养流程是系统的另一个核心部分。用户可以通过系统提交领养申请，填写自己

的姓名、联系方式、住址等信息。系统会对这些申请进行审核。管理员可以看到所有待审核的申请，并决定是否通过。一旦审核通过，该宠物的状态就会变为“不可领养”，防止重复申请。同时，系统支持批量处理，可以一次审核多个申请。

识别匹配业务是本项目的核心设计。用户宠物丢失后，可以在平台上上传自己宠物的照片，系统调用训练好的宠物匹配模型，与已有的宠物档案库进行比对，返回相似的宠物信息列表。以提高救助效率，帮助走失宠物更快找到主人。

在社区互动方面，户可以在社区中发布文章，分享养宠经验，或者提出问题。其他用户可以评论这些文章，与其它用户在评论区互相交流。系统还开发了点赞和收藏等功能，可以让好的内容更容易被发现。此外，系统也允许用户上传文件，支持数据的导入导出，方便资料共享。系统用例图如图 5 所示。

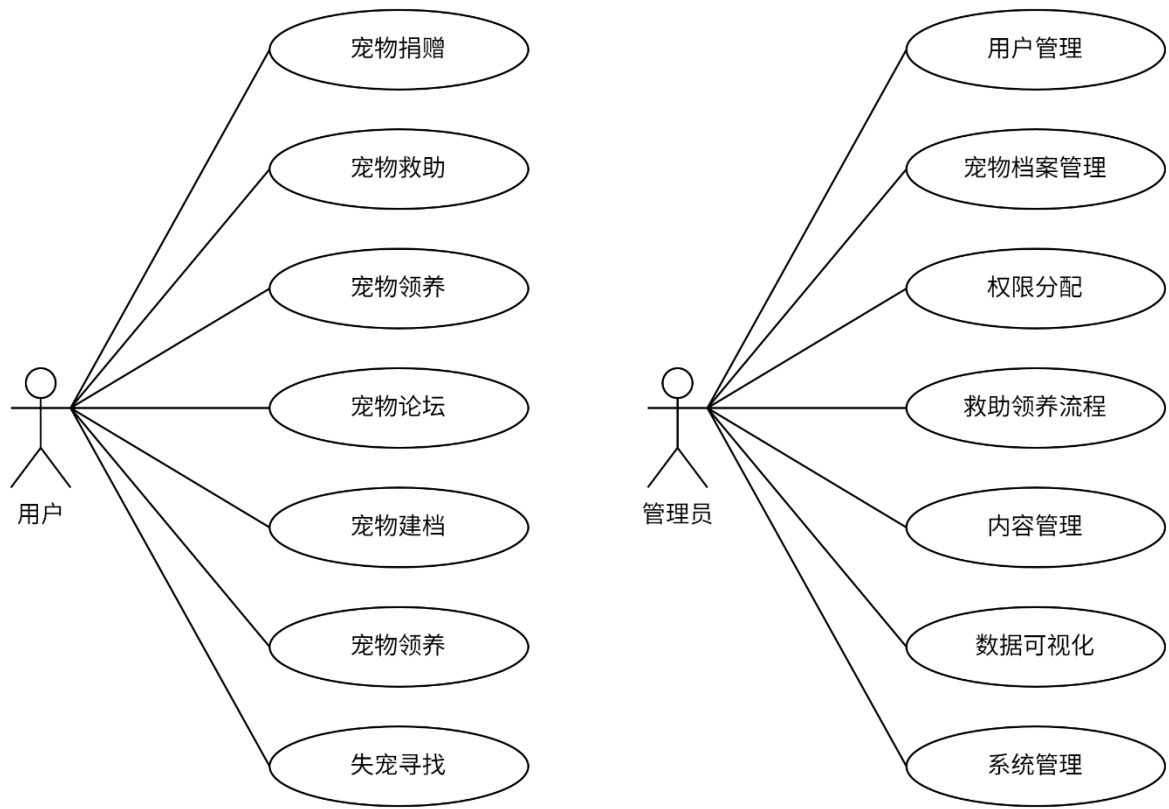


图 5 系统用例图

Fig.5 System use case diagram

在数据管理上，系统支持分页查询、搜索、新增、编辑和删除等操作。系统还具备完善的权限控制机制。普通用户只能访问公开资源，而管理员则可以操作所有功能。例如，只有管理员可以创建宠物档案，修改宠物状态。系统通过 JWT 验证用户身份，确保不同角色的用户只能访问自己有权限的资源。

系统涵盖宠物救助的主要业务流程。从宠物建档到领养申请，再到图像识别和社区互动，每一项功能都围绕着救助和领养展开。同时，系统应该要具有良好的可扩展性，方便以后新的业务模块的拓展。系统整体结构应该清晰简便，用户友好。系统功能结构图如图 6 所示。

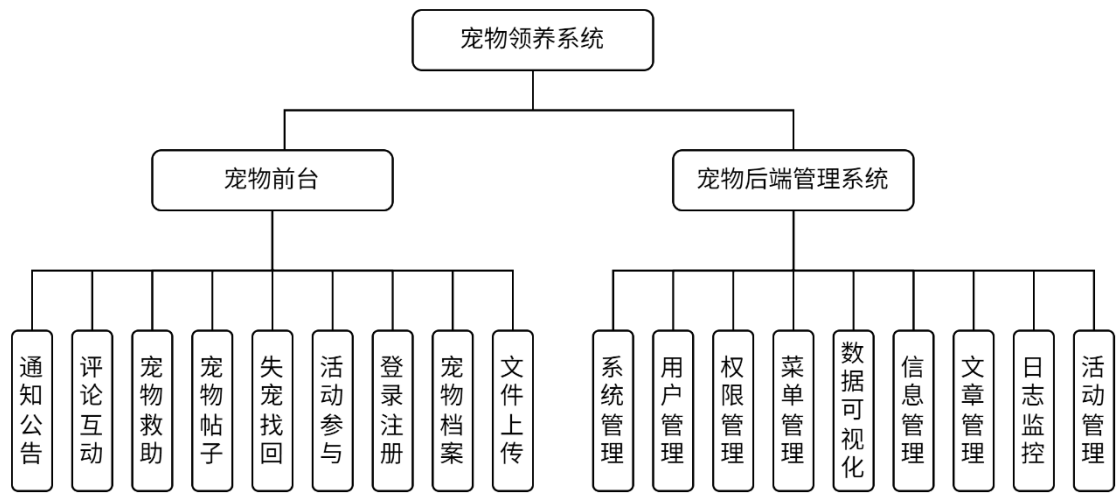


图 6 系统功能结构图

Fig.6 System function structure diagram

3.1.2 非功能性需求

该系统在设计和实现时需要满足一系列非功能性需求，以确保其能够在实际运行中表现稳定、易于扩展，并为用户提供良好的操作体验。首先，在性能方面，系统应具备快速响应用户请求的能力，尤其是在处理大量数据或高并发访问时仍能保持较高的效率。

考虑到用户体验。前端的页面应要简洁直观，操作流程清晰易懂，让不同技术水平的用户都可以轻松上手。对于涉及文件上传、数据导入导出等具体的功能，也应该向用户提供友好的提示和反馈，让用户清楚当前操作的状态和结果。

稳定性与兼容性也是系统不可忽视的一点，我们的系统要可以在不同的浏览器环境和设备上正常运行，同时还要在长时间运行过程中保持较低的故障率。这些非功能性需求将直接影响系统的实用性与长期发展潜力，因此在开发过程中应给予足够的重视。

3.2 系统总体架构设计

本系统采用前后端分离架构，前端使用 Vue 框架实现页面展示与交互逻辑，通过 Axios 发起对后端的请求。后端基于 Spring Boot 构建，向前端提供 RESTful API 接口，通过 RestController 和路由配置接收处理前端请求，两者之间通过 HTTP 协议进行数据

通信。系统整体架构图如图 7 所示。

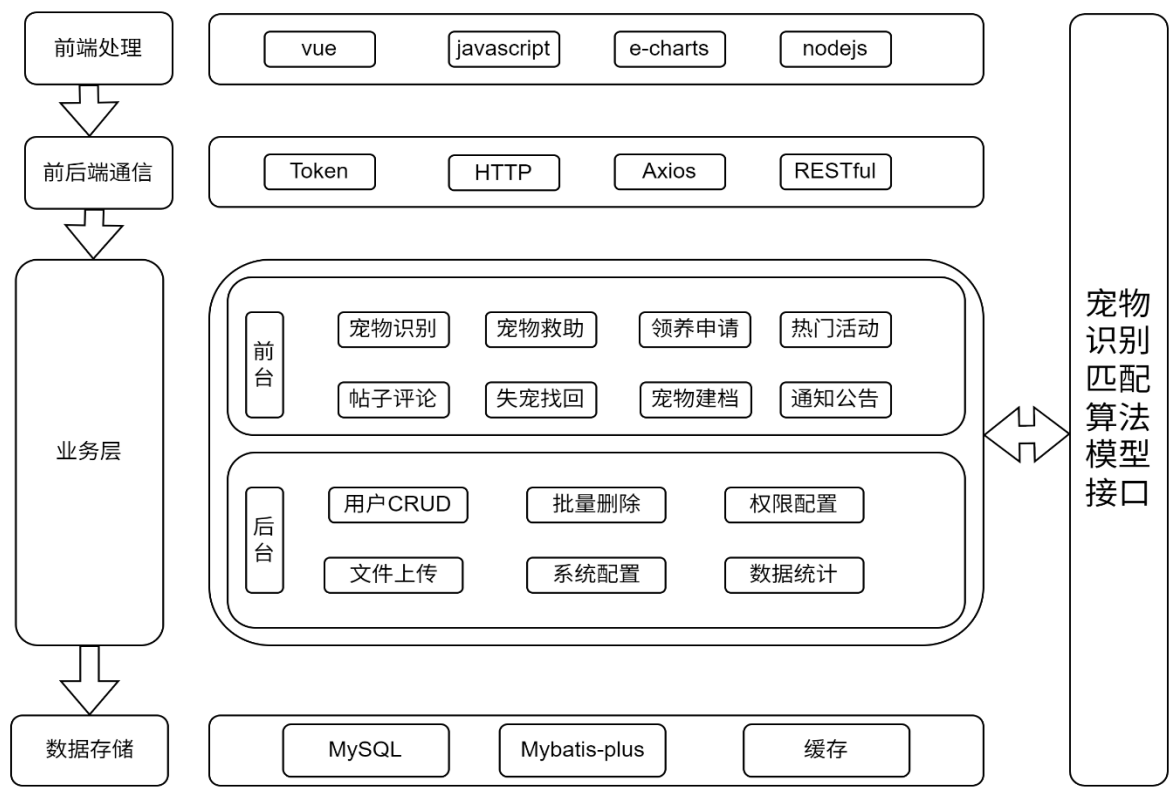


图 7 系统整体架构图

Fig.7 Diagram of the overall architecture of the system

3.3 数据库设计

3.3.1 数据库概要设计

宠物领养系统的数据库需要记录用户信息、动物数据、领养流程和系统管理内容。先从用户相关表开始设计，用户表用来保存每个使用系统的账号信息，比如用户名、登录密码、姓名、联系方式和地址等个人资料。角色表定义不同权限等级，例如普通用户、管理员和志愿者，菜单表列出系统功能模块，通过角色菜单表把权限分配到具体用户类型。

动物信息是核心部分，动物表需要包含种类、品种、年龄、性别、健康状态、照片和所在救助站等信息。领养表记录每只动物的领养状态，通过动物编号关联到对应的动物记录。当用户申请领养时，领养申请表会存储用户 ID、动物 ID、申请时间和申请理由，同时关联用户表和动物表，完整记录申请流程。

救助功能需要单独处理，救助机构表保存合作单位的信息，包括机构名称、联系人、地址和联系方式。救助记录表记录每次救助的详细情况，比如救助日期、动物状况和救助人员，通过用户 ID 关联提交救助的个人或机构，方便后续跟踪管理。

其他辅助功能通过独立表实现，活动表存储线下领养活动的时间、地点和内容。文

章表保存系统发布的科普文章或公告，通过用户 ID 和文章 ID 建立关联。走失信息表帮助用户发布丢失宠物的信息，包含宠物特征、丢失地点和时间等。

整个数据库采用关系型结构设计，每个表都有唯一标识的主键字段，通过外键建立表之间的联系。字段类型根据实际内容设置，比如日期字段用日期格式，金额字段用数值类型。为常用查询字段添加索引，例如按动物种类或领养状态查询时能快速获取结果。这种设计既能满足业务流程需要，又能保证数据安全和系统运行效率。系统 ER 图如图 8 所示。

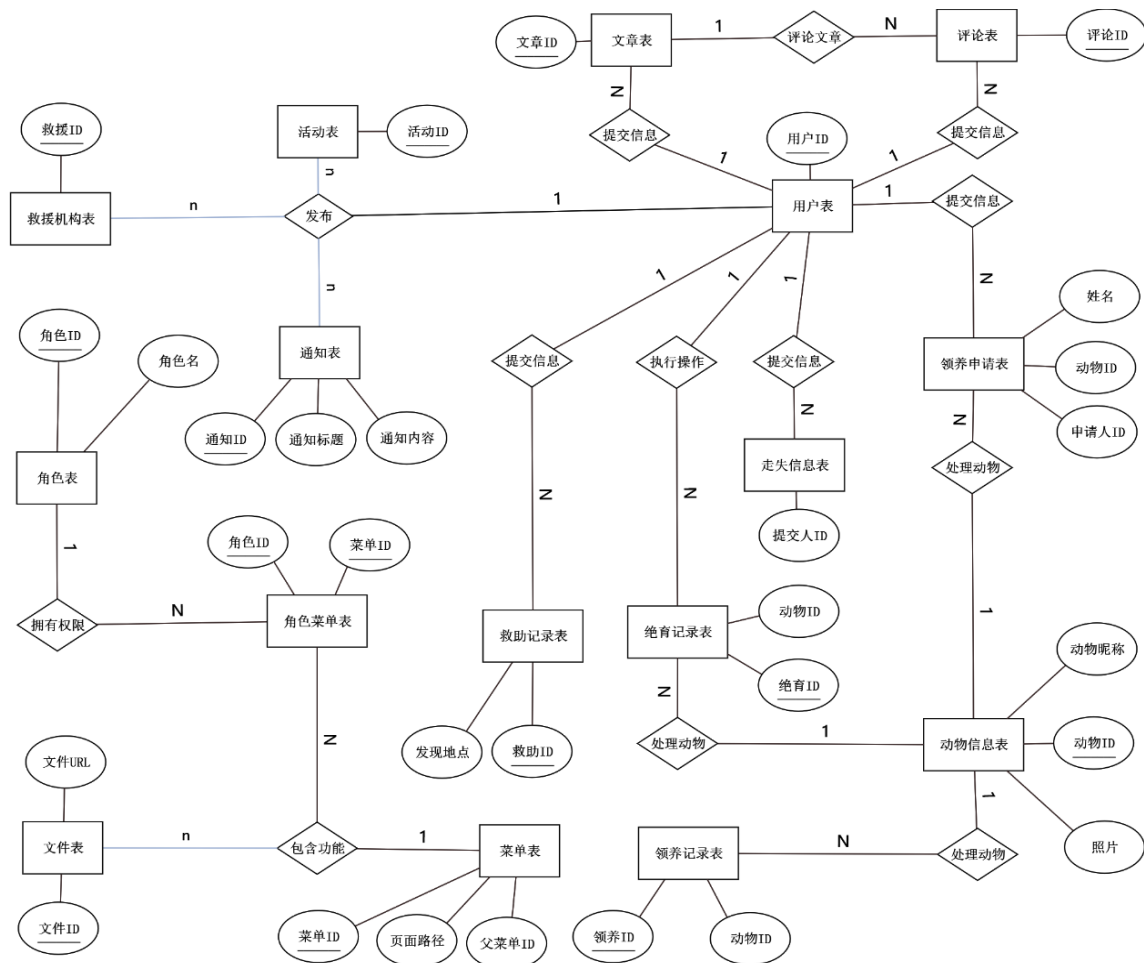


图 8 系统 ER 图

Fig.8 System ER diagram

3.3.2 数据库表设计

系统的数据库主要是基于宠物建档与领养的业务需求来设计的，包含宠物档案表、流浪动物表、领养申请表及用户表等。宠物档案表用于记录宠物的昵称、绝育状态等特征，通过 image_id 关联走失宠物表。通过 sys_role_menu 实现用户权限的分级管理，以支持管理员审核与普通用户申请。数据库表如表 1 到表 15 所示。

表 1 活动表

Table1 Activity Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	标题
content	text	-	否	内容
time	varchar	255	否	时间
img	varchar	255	否	封面
num	int	-	否	报名人数
address	varchar	255	否	地址
org	varchar	255	否	组织

表 2 领养表

Table2 Adopt Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
animal_id	int	-	否	流浪动物 id
adopt_status	varchar	255	否	领养状态
address	varchar	255	否	领养地点

表 3 宠物表

Table3 Animal Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
nickname	varchar	255	否	动物名字
sex	varchar	255	否	动物性别
type	varchar	255	否	种类
age	varchar	255	否	年龄
img	varchar	255	否	动物照片
address	varchar	255	否	活动范围
status	varchar	255	否	身体状态
is_adopt	varchar	255	否	是否被领养

表 4 申请表

Table4 Application Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	姓名
sex	varchar	255	否	性别
age	int	-	否	年龄
experience	varchar	255	否	养宠经验
pet	varchar	255	否	宠物
phone	varchar	255	否	联系方式
reason	varchar	255	否	领养理由
state	varchar	255	否	状态
animal_id	int	-	否	动物 id
user_id	int	-	否	用户 id

表 5 文章表

Table5 Article Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	文章名
content	text	-	否	文章内容
user_id	int	-	否	用户 id
time	varchar	255	否	时间
user	varchar	255	否	用户
comment	text	-	否	评论

表 6 科普文章

Table6 Article_kp Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	标题
content	text	-	否	内容
time	varchar	255	否	时间
img	varchar	255	否	封面
read	int	-	否	阅读数

表 7 评论表

Table7 Comment Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
content	varchar	255	否	回复内容
user	varchar	255	否	回复人
time	varchar	255	否	回复时间
pid	int	-	否	父 id
article_id	int	-	否	文章 id
type	int	-	否	type

表 8 捐赠表

Table8 Denotes Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	捐款人
goods	varchar	255	否	捐赠物资
time	varchar	255	否	捐款事件
address	varchar	255	否	捐款地址
Number	int	-	否	捐款数量

表 9 喂养表

Table9 Feed Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
img	varchar	255	否	环境照片 1
img2	varchar	255	否	环境照片 2
img3	varchar	255	否	环境照片 3
address	varchar	255	否	地址
type	varchar	255	否	宠物类型
information	text		否	相关描述

表 10 走失表

Table10 Lost Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
nickname	varchar	255	否	走失宠物名字
type	varchar	255	否	种类
sex	varchar	255	否	性别
person	varchar	255	否	联系人
phone	varchar	255	否	联系方式
img	varchar	255	否	宠物照片
lost_time	datetime	-	否	丢失时间
lost_place	varchar	255	否	丢失地点

表 11 通知表

Table11 Notice Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	名称
content	varchar	255	否	内容
time	varchar	255	否	时间

表 12 宠物档案表

Table12 Pet_archive Table

字段名称	类型	长度	主键	备注
id	bigint	-	是	主键
nickname	varchar	255	否	宠物昵称
pet_id	varchar	255	否	宠物数据库 ID
found_time	datetime	-	否	捡到时间
found_location	varchar	255	否	捡到地点
pet_description	text	-	否	宠物描述
pet_type	int	-	否	宠物类型：0-狗，1-猫

表 13 救助表

Table13 Rescue Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
name	varchar	255	否	名称
pet_description	text	-	否	宠物描述
addres	varchar	255	否	地址
img	varchar	255	否	照片
person	varchar	255	否	联系人
phone	varchar	255	否	联系方式
information	varchar	255	否	相关描述

表 14 救助站表

Table14 Salvation Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
information	varchar	255	否	情况描述
img	varchar	255	否	现场照片
address	varchar	255	否	地点
time	varchar	255	否	发现时间
person	varchar	255	否	联系人
type	varchar	255	否	动物类型

表 15 系统用户表

Table15 Sys_user Table

字段名称	类型	长度	主键	备注
id	int	-	是	id
username	varchar	255	否	用户名
password	varchar	255	否	密码
nickname	varchar	255	否	昵称
email	varchar	255	否	邮箱
create_time	timestamp	-	否	创建时间
avatar_url	varchar	255	否	头像
birth	varchar	255	否	出生年月

4 宠物识别匹配算法设计与实现

4.1 数据准备与预处理

为了让识别模型更高效的处理学习数据，对于图像数据的预处理是必不可少的。

4.1.1 宠物数据集构建

构建宠物数据集是宠物识别算法中的关键一步构建宠物数据集是宠物识别算法的关键。我们主要收集猫、狗等常见宠物类型，覆盖多个常见品种以提高识别全面性。同时对每张图片中宠物的毛色、耳朵形状、眼睛颜色、体型等特征进行分析，助模型区分品种，如柯基短腿、布偶猫蓝眼睛等。还尽量涵盖不同角度、光照条件下的图片，让模

型在实际应用中面对各种情况时保持良好识别效果。

最后收集数据集包含猫狗多个品种的图像，分为训练集（train）和验证集（val），总计涵盖数十个宠物类别,如表 16 所示。每个类别的训练集样本数量约为 360 张左右，每张图片都会打上详细的标签，包括宠物的种类、品种和主要特征，如图 9 所示。这些标签帮助模型理解输入数据，并在训练过程中不断优化识别能力。

表 16 宠物数据集

品种	图片数量	品种	图片数量
Abyssinian	359	Persian	360
Bengal	360	Ragdoll	360
Birman	360	Russian	360
Bombay	360	Siamese	360
British	360	Sphynx	360
Egyptian	360	american	520
Maine	360	basset	360
beagle	361	boxer	360
chihuahua	360	english	520
german	360	great	360
havanese	360	japanese	360
keeshond	360	leonberger	360
miniature	360	newfoundland	360
pomeranian	360	pug	360
saint	360	samoyed	360
scottish	359	shiba	360
staffordshire	352	wheaten	360
yorkshire	360		

通过这样的构建方式，我们收集了国内外各个种类的猫犬照片超过四万张，数据集不仅覆盖面广，而且具备高质量的标注信息，为后续模型训练提供了坚实基础。

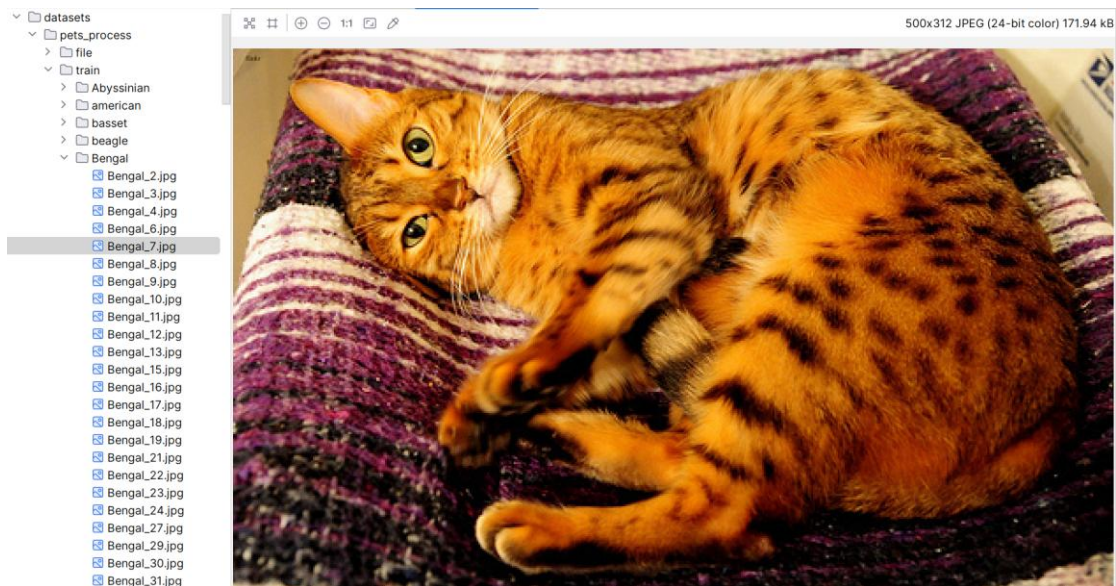


图 9 宠物数据集图

Fig.9 Pet dataset graph

4.1.2 数据增强与清洗

进行数据增强和清洗操作可以提高模型性能。对于实际环境中可能存在的光线、遮挡等问题，可以对图片进行一些预处理来增强模型的适应能力。

数据增强的主要目的是让训练数据更加多样。我们可对图片进行旋转、翻转、缩放等操作。这样可以让模型在面对真实场景时更稳定地识别宠物。

相关代码：

数据预处理

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop((224,224)), # 随机裁剪并缩放到 224x224
    transforms.RandomHorizontalFlip(),      # 随机水平翻转
    transforms.RandomVerticalFlip(),        # 随机垂直翻转
    transforms.ToTensor(),                  # 将图像转换为 Tensor
])

val_transform = transforms.Compose([
    transforms.Resize((224,224)),           # 缩放到 224x224
    transforms.ToTensor(),                  # 将图像转换为 Tensor
])
```

针对遮挡问题，我们引入了“随机遮挡”技术。具体做法是在图像上随机位置添加

一块或几块颜色相近的小方块。这种技术可以帮助模型学会即使部分身体被挡住也能做出正确判断。

应用随机遮挡代码：

```
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 1.7),
value='random', inplace=False),
])
```

经过上面的数据处理，清洗去除掉一些质量差或不合适的图片，避免谬误数据影响模型的学习效果。最后将所有图片调整为统一尺寸，并进行标准化处理。

通过这些方法，我们可以有效解决光照、遮挡等问题带来的影响。整个过程虽然简单，但对模型的表现有明显帮助。

4.2 基于深度学习的宠物特征提取模型

4.2.1 模型选型与改进

在宠物识别训练算法中，模型选型和改进方法的选择对最终效果有直接影响。我们首先选择了 ResNet50 作为基础模型。这是因为 ResNet50 在图像识别任务中表现良好，如表 17 所示，ResNet50 与其它模型的对比，能够有效提取图像特征。

表 17 模型对比表

Table17 Model comparison					
模型	深度	Top-5 错误率	参数规模	计算效率	主要优势
ResNet50	50	5.3%	25.6M	高	残差连接、深层优化
VGG16	16	7.3%	138M	低	结构简单、特征表达强
InceptionV3	48	4.2%	23.8M	中	多尺度特征融合
简单 CNN	<10	15 - 20%	<1M	极高	训练速度快、资源需求低

接下来，我们需要对 ResNet50 做出一些改进。我们在模型的输出层之前加入了一个全连接层来优化宠物匹配的准确率，这样可以让模型更准确地识别不同种类的宠物。

总而言之我们在原有的经典模型的基础上做了一些简单但有效的改进。通过上面的步骤，经过大量的高质量数据训练后可以得到一个既高效又准确的宠物识别模型。

4.2.2 迁移学习与模型微调

在宠物识别训练算法中，模型迁移学习和微调是非常重要的步骤。这些方法可以

帮助我们利用已有的模型知识，快速适应新的任务。

迁移学习可复用预训练模型特征提取能力提升小数据集任务性能。关键方法有：特征提取，冻结卷积层、替换训练全连接层，适用于小数据集；微调，用分层解冻、调整学习率和正则化，适合细粒度任务；PEFT 如 Adapter、Prefix-tuning，适用资源有限场景。

接下来需要对 ResNet50 进行微调。具体来说，我们改变了模型的全连接层，使其适合我们的宠物分类任务。原来的全连接层被替换成了一个新的全连接层，这个新层的输出节点数等于我们宠物类别的数量。这样做可以让模型更好地适应我们的数据。

在训练过程中，我们不仅更新了新添加的全连接层，还对整个模型的参数进行了微调。通过调整模型中所有层的权重，以优化其在宠物识别任务上的表现。这种做法可以进一步提升模型的准确率。通过这些方法，我们成功构建了一个可靠的宠物识别系统。

4.3 宠物特征匹配算法设计

在宠物特征匹配算法的设计中，我们首先需要提取每只宠物的特征向量。这个过程是为了让计算机能够理解并比较不同宠物之间的相似性。

首先通过加载已训练好的模型 ResNet50，并对它的最后一层进行修改，使其适合宠物分类任务。然后，我们用这个模型来处理宠物图片，得到每个宠物的特征向量。特征向量可以理解为一张图片的数字表示，它包含了图片中的关键信息。

随后进行特征向量的相似度计算。以判断两只宠物是否属于同一品种。我们使用了一种叫做余弦相似度的方法来进行计算。余弦相似度是一种常用的相似度度量方法，它可以衡量两个向量在方向上的相似度。

我们首先获取两只宠物的特征向量。然后，我们计算这两个向量的余弦相似度，部分公式如（4）所示。余弦相似度的值范围在-1 到 1 之间，值越接近 1，表示两个向量越相似。因此，如果两只宠物的特征向量余弦相似度很高，我们就认为它们很可能是同一种宠物。

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (4)$$

基于余弦相似度，可以设置一个相似度阈值来判断两只宠物是否为同一只宠物。如果相似度大于这个阈值，我们就认为这两只宠物是同一品种。

此外，我们还建立了一个数据库来存储每只宠物的特征向量。这样在进行新宠物

的识别时，可以做到快速地与数据库中的特征向量进行比较，找到最相似的宠物。

基于特征向量计算余弦相似度，我们可以高效地判断宠物的特征并进行匹配。通过不断优化模型和算法，我们可以进一步提高识别的准确率，为宠物救助管理系统提供更好的支持。

5 宠物领养社区平台实现

5.1 核心功能模块设计

5.1.1 流浪宠物识别匹配模块设计

宠物识别匹配模块是本项目中的一个关键功能，用于通过上传的宠物图片识别宠物的品种，并进行相关的档案管理和查找。该模块主要涉及两个核心接口：一个是获取识别所需的访问令牌，另一个是实际执行宠物识别和匹配的功能。

识别页面设计如图 10 所示：



图 10 宠物识别页面线框图

Fig.10 Pet identification page wireframe

在实现中，首先需要获取访问系统的令牌，这是为了确保操作的安全性和权限控制。这部分通过 `PetTokenController` 类来处理，其提供的 `fetchToken` 方法可以生成一个访问令牌，客户端可以在后续请求中携带这个令牌来验证身份，从而获得服务端的认可。如果令牌获取失败，则返回错误信息，否则返回成功结果以及对应的令牌。

接着是具体的宠物识别接口，它负责接收上传的图像文件并执行识别任务。这一功能由 `PetArchiveController` 类实现，其中的 `identifyImage` 方法负责处理上传的文件，将其保存为临时文件后发送至指定的后端服务。在上传过程中，系统会构建一个包含文件数据的 HTTP 请求，并等待服务器的响应结果。一旦收到响应，系统会从中提取出识别到的宠物类型和文件路径等信息，以供后续使用。若上传或识别过程中出现异常，将抛出相应的错误提示。

对于识别接口的设计，它不仅关注图像本身，还允许传递额外参数如宠物 ID、识别类型等，以便于进行更精确的匹配。此接口设计了专门的请求对象 `VerifyRequest`，封装了这些参数，并调用对应的服务层完成识别逻辑。业务流程如图 11 所示

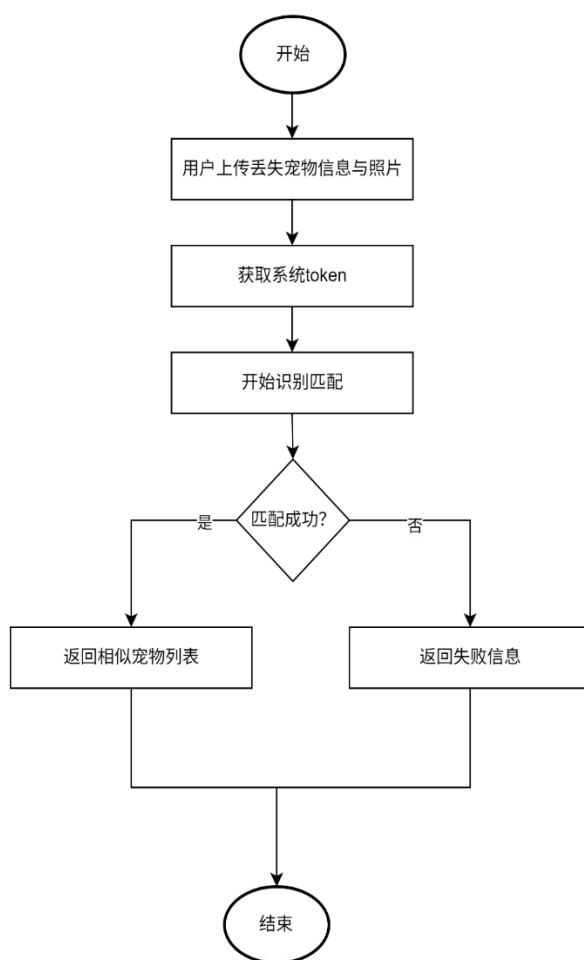


图 11 宠物识别匹配流程图

Fig.11 Pet Identification Matching Flow Chart

接口设计: `POST /api/pet/verify`, 参数: `imageUrl`、`petId`、`petType`、`recogType`、`token`。
功能: 执行宠物身份验证, 确保上传的图片与宠物档案匹配。
`POST /api/pet/recognize`。
参数: `imageUrl`、`petType`、`recogType`、`token`、`topK`。功能: 识别宠物图片并返回最可

能的 K 个宠物品种及其置信度，以方便用户根据宠物的 id 精确匹配到宠物。

5.1.2 宠物建档与领养管理模块设计

宠物档案管理模块主要用于存储和管理流浪宠物的相关数据。系统会记录每只宠物的关键信息，包括宠物 ID、昵称、性别、出生日期、是否绝育、验证状态、图片地址、分组 ID、备注、宠物类型等字段，可以全面地描述出一只宠物的基本情况，方便后续业务的查询和更新操作。

宠物建档线框图设计如图 12 所示，宠物领养线框图设计如图 13 所示。

流浪宠物建档

收养人姓名

收养人电话

捡到时间

03 / 10 / 2014

捡到地点

宠物描述

宠物类型

☒ 狗

☒ 猫

宠物性别

☒ 公

☒ 母

宠物照片



已建档宠物

图 12 宠物建档线框图

Fig.12 Pet profiling wireframe

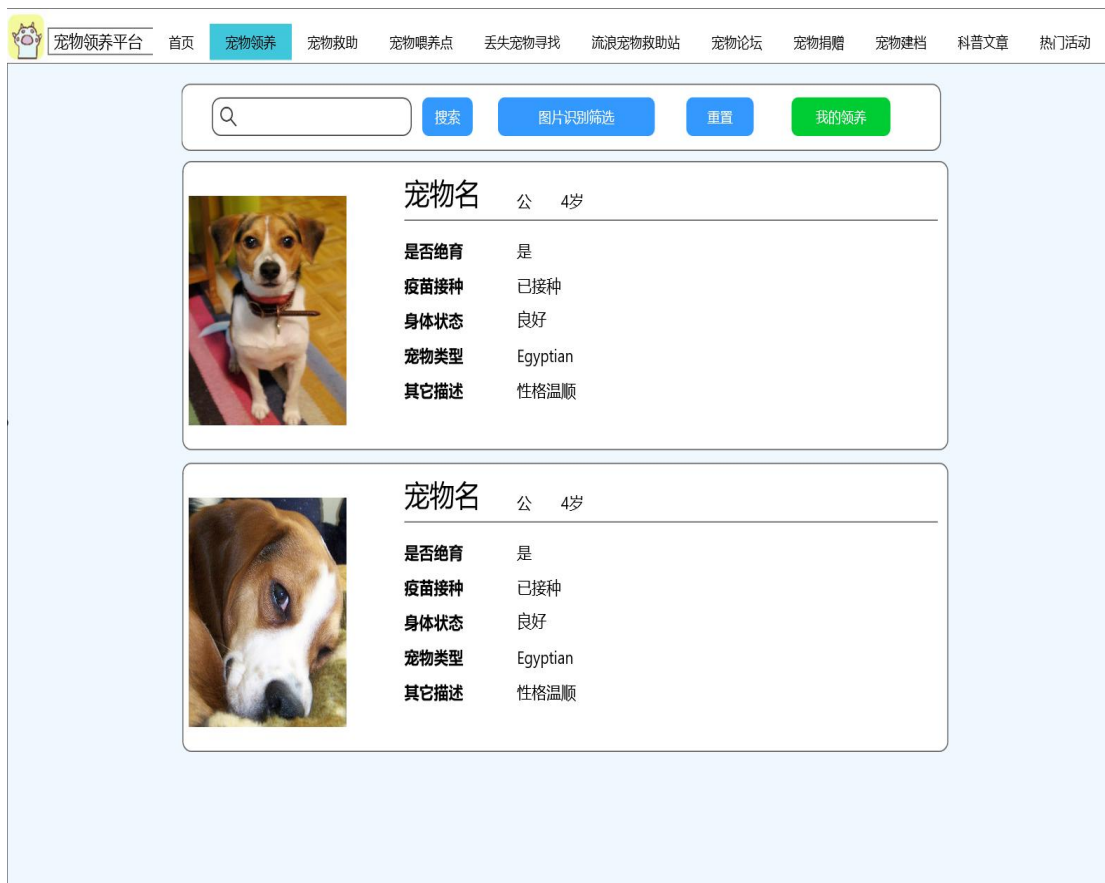


图 13 宠物领养页面线框图

Fig.13 Pet adoption page wireframe

领养模块构建动态交互体系，前端首页通过宠物列表展示可领养宠物卡片。用户申请时，JWT 令牌会自动绑定账户信息，以减少重复的输入，提高系统的性能。审核流程则采用分级机制：普通用户的申请由管理员进行初审，特殊品种或高龄宠物则还需主管复核。数据库通过事务锁确保领养状态变更与申请记录写入的原子性，结合 Elasticsearch 实现多条件检索优化，`adopt_status` 字段建立覆盖索引提升查询效率。

系统基于前后端分离的架构设计，前后端通过统一的接口进行交互。所有接口返回统一 JSON 格式，包含明确状态码，如 200 成功、400 参数错误及业务提示，便于前端快速解析并且反馈给用户。跨域访问通过 CORS 策略实现，配置白名单控制资源访问权限。技术实现上，前端采用 Axios 封装请求拦截器，自动申请并且携带 Token；后端通过 Spring Boot 全局异常处理器捕获错误，返回结构化的错误信息。数据库设计遵循资源导向原则，宠物档案与领养申请分别对应独立端点（`/pets`、`/adoptions`），支持标准 HTTP 方法进行增删改查操作。业务流程如图 14 所示。

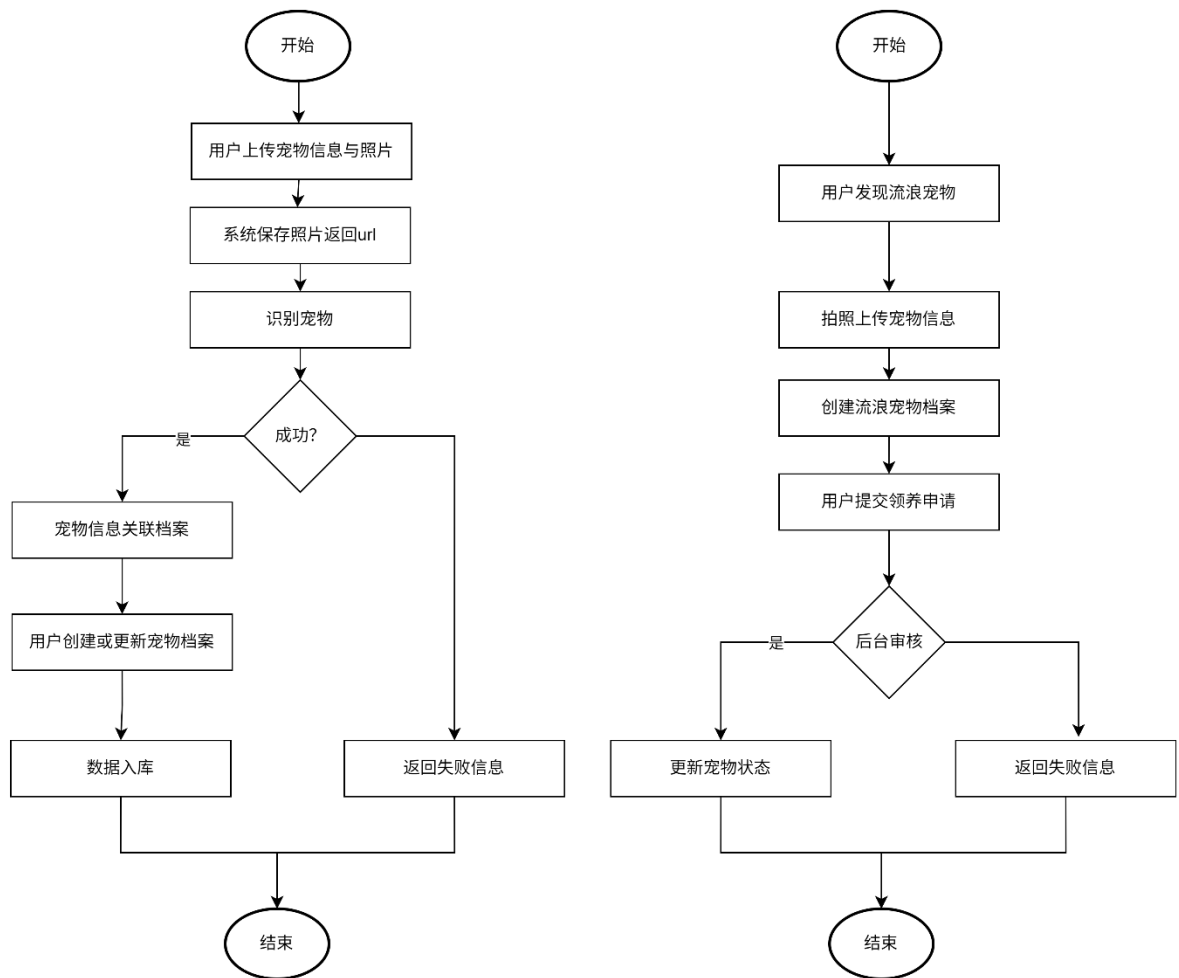


图 14 宠物建档与流浪宠物领养程序流程图

Fig.14 Flowchart of pet filing and stray pet adoption procedures

接口设计：POST /api/pet/create 这个接口用于创建一个新的宠物档案。用户需要提供宠物的基本信息和图片。参数包括：petId、nickname、gender、dateOfBirth（、birthControlStatus、token，系统会将这些信息保存到数据库，并调用第三方 API 进行宠物面部识别或鼻纹识别，生成一个唯一的宠物 ID。GET /api/pet/archives/list 这个接口用来获取所有宠物的档案信息，支持分页查询。参数包括：query、pageNum、pageSize，返回值是一个包含宠物档案列表的分页数据，包括总记录数和当前页的数据。

5.1.3 社区互动模块设计

社区互动功能主要通过发帖和评论来实现，用户可以发布交流贴和在其它帖子下面进行评论。管理员在后台可以控制评论，对评论进行新增、删除、批量删除等管理操作。同时，系统还支持查看某一特定文章下的所有评论，并将这些评论按层级结构展示出来，使得用户能够方便地了解讨论内容。

社区页面设计，如图 15 所示：



图 15 宠物社区线框图

Fig.15 Pet community wireframe

当用户提交一条新的评论时，系统会自动记录该用户的昵称及评论的时间。平台管理员可以删除评论，防止不良言论产生不良影响。对于每条评论，可以是单独的一条评论，也可以是其它评论的回复，这样就形成了一个父子级联的评论体系。这种设计可以让整个社区内的对话更加有序且具有针对性。

为了确保系统的稳定性和安全性，在处理评论数据时，程序采用了数据库持久化存储方案将相关数据存到本地数据库中，这样既保证了信息不会丢失并且可以快速检索到之前的历史记录。每当有新的评论产生或者已有评论被修改时，都会对数据库进行更新，比如刷新页面显示的数据列表。业务流程如图 16 所示。

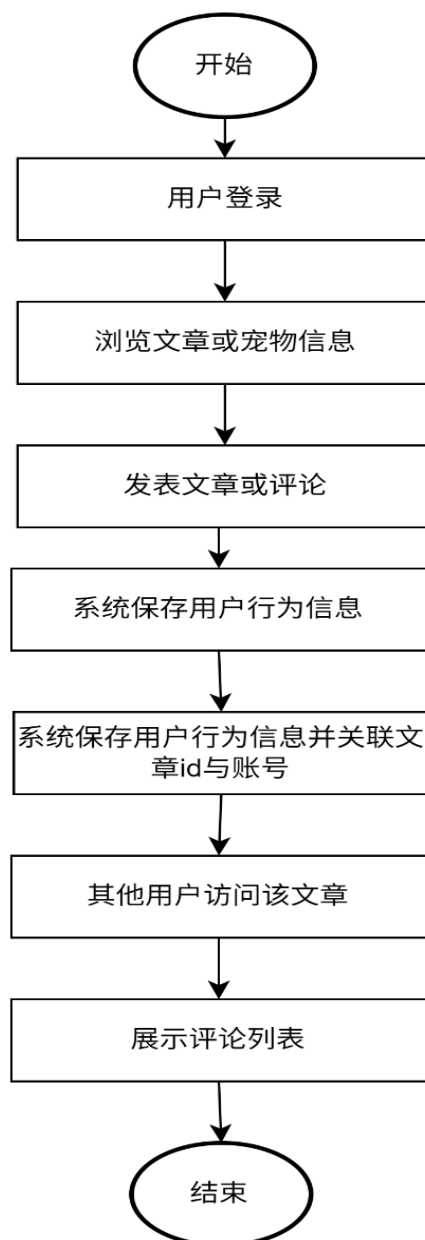


图 16 社区互动程序流程图

Fig.16 Community Interaction Program Flow Chart

接口设计：（1）发布活动评论：请求路径/api/activity/comment，方法: POST，参数: token、activityId、content、userId，用户对指定活动发表评论。系统会验证用户身份，并将评论内容存储到数据库。（2）发布宠物帖子：请求路径/api/pet/post，方法: POST 参数: token、title、content、images。用户创建宠物主题的社区帖子。支持上传图片，生成后返回帖子 ID。（3）获取单个帖子详情：请求路径: /api/pet/post/{postId} 方法: GET 参数: postId。查看完整帖子内容，包含正文、图片、评论数量和点赞总数。

5.2 核心功能模块实现

5.2.1 流浪宠物识别匹配模块实现

用户上传宠物图片后，系统先检查图片是否存在。接着将图片文件与宠物类型、识别方式等参数打包成网络请求。然后通过互联网将请求发送到识别服务器。服务器处理完成后，系统接收返回结果并解析数据。若结果中包含宠物编号列表，则显示匹配成功的宠物信息；若列表为空，则提示未找到匹配对象。整个过程通过异常捕获机制确保稳定性，本地日志会记录完整请求响应数据用于调试。

识别页面实现，如图 17 所示：

1:N 宠物查找匹配

匹配成功

宠物照片

图片上传成功!



重新上传

支持 JPG/PNG/JPG/BMP, 最大 2MB

宠物类型

☒ 狗 ☐ 猫

识别类型

☐ 鼻纹 ☒ 正脸

返回数量

5

开始查找

宠物照片	宠物ID	置信度
	b8901208-2e2c-11f0-2vjeM9y1l	0.7912902974140593

图 17 宠物识别界面

Fig.17 Pet identification interface

核心功能伪代码：

```
def recognize_pet(image_path, threshold=0.9):  
    breed, new_feature = predict_and_extract_features(image_path) # 提取特征向量  
    all_pets = get_all_features() # 获取数据库中所有宠物特征  
    for pet_id, pet_breed, stored_feature in all_pets:  
        similarity = calculate_similarity(new_feature, stored_feature) # 计算相似度  
        if similarity > threshold and breed == pet_breed:  
            return pet_id, pet_breed # 成功匹配，返回宠物 ID 和信息  
    pet_id = add_pet(breed, new_feature) # 未找到匹配项，新建记录  
    return pet_id, breed # 返回新宠物 ID 和品种
```

5.2.2 宠物建档与领养管理模块实现

如果用户捡到了流浪宠物，可以在平台上传宠物的图片并填写相关信息对宠物进行建档，系统拿到数据后会先将数据存入本地数据库。接着调用第三方算法接口分析图片特征，生成唯一宠物 ID 并绑定用户信息。领养申请时，用户提交个人信息和宠物 ID，系统验证资料完整性后将申请记录存入数据库。审核人员通过后台查看申请列表，人工核对后更新领养状态。已领养宠物的信息会同步更新到动物档案表，标记为“不可再次领养”。

宠物建档界面实现如图 18 所示，宠物领养界面如图 19 所示。

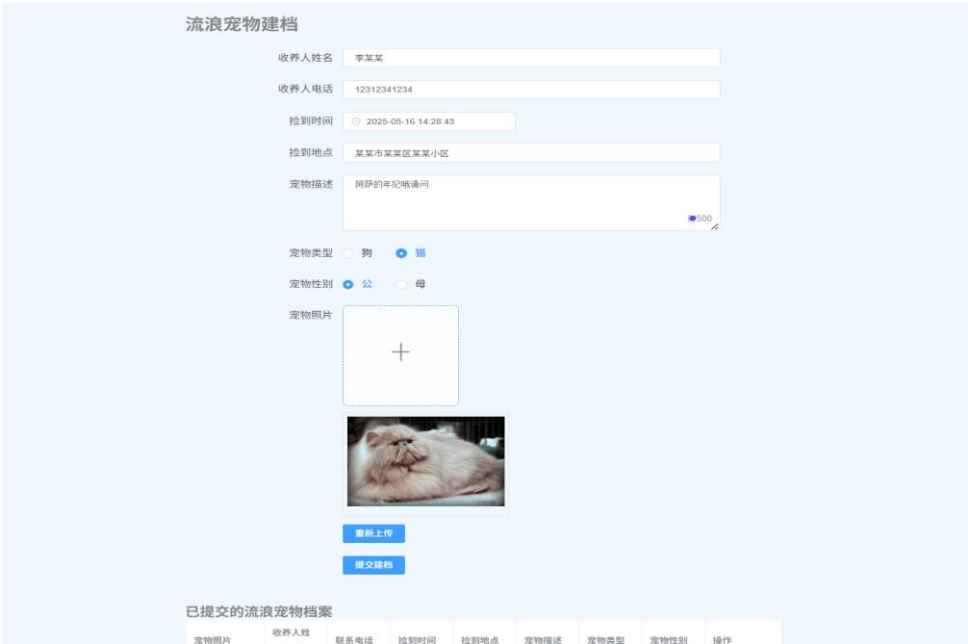


图 18 宠物建档界面

Fig.18 Pet profile interface



图 19 宠物领养界面

Fig.19 Pet adoption interface

核心伪代码:

// 伪代码: 宠物匹配识别

```
public Result recognizePet(String imageUrl, Integer petType, String token) {
    File file = downloadImage(imageUrl); // 下载图片到本地
    if (!file.exists()) return error("文件不存在");
    RequestBody body = buildMultipart(file, petType, token); // 构建请求体
    Response response = sendPostRequest(RECOGNIZE_URL, body); // 发送请求
    if (response.success) {
        return success(response.data); // 返回识别结果
    } else {
        return error("识别失败");
    }
}
```

// 领养申请接口

```
public Result applyForAdoption(PetArchiveDTO dto) {
    User user = verifyUserLogin(dto.getToken()); // 1. 校验用户是否已登录
    PetArchive petArchive = convertToEntity(dto); // 2. 创建本地宠物档案
```

```
boolean saved = petArchiveService.save(petArchive); //3.创建宠物面部识别档案
Result thirdPartyResult = callExternalApi(dto);
if (thirdPartyResult.isSuccess()) { // 4. 如果成功，更新数据库记录
    updatePetArchiveWithExternalData(petArchive, thirdPartyResult.getData());}
return thirdPartyResult; // 5. 返回结果给前端}
```

5.2.3 社区互动模块实现

用户发布文章时，系统首先需要拿到图片和内容，将数据存入数据库，然后同步展示在前端页面。其他用户可浏览并做出评论，系统验证身份后关联评论与文章。由管理员定期更新公告，然后用户可以看到最新消息。当发现走失宠物时，用户提交信息后由后台审核展示。所有功能通过 RESTful API 与前端交互，使用 JWT 验证身份，关键操作记录日志。数据库表包含文章、评论、公告等字段，支持分页查询和图文混排显示。

宠物社区论坛界面如图 20 所示，宠物科普文章界面如图 21 所示。



图 20 宠物社区论坛界面

Fig.20 Pet community forum interface



图 21 宠物科普文章界面

Fig.21Pet science article interface

宠物活动评论界面如图 22 所示

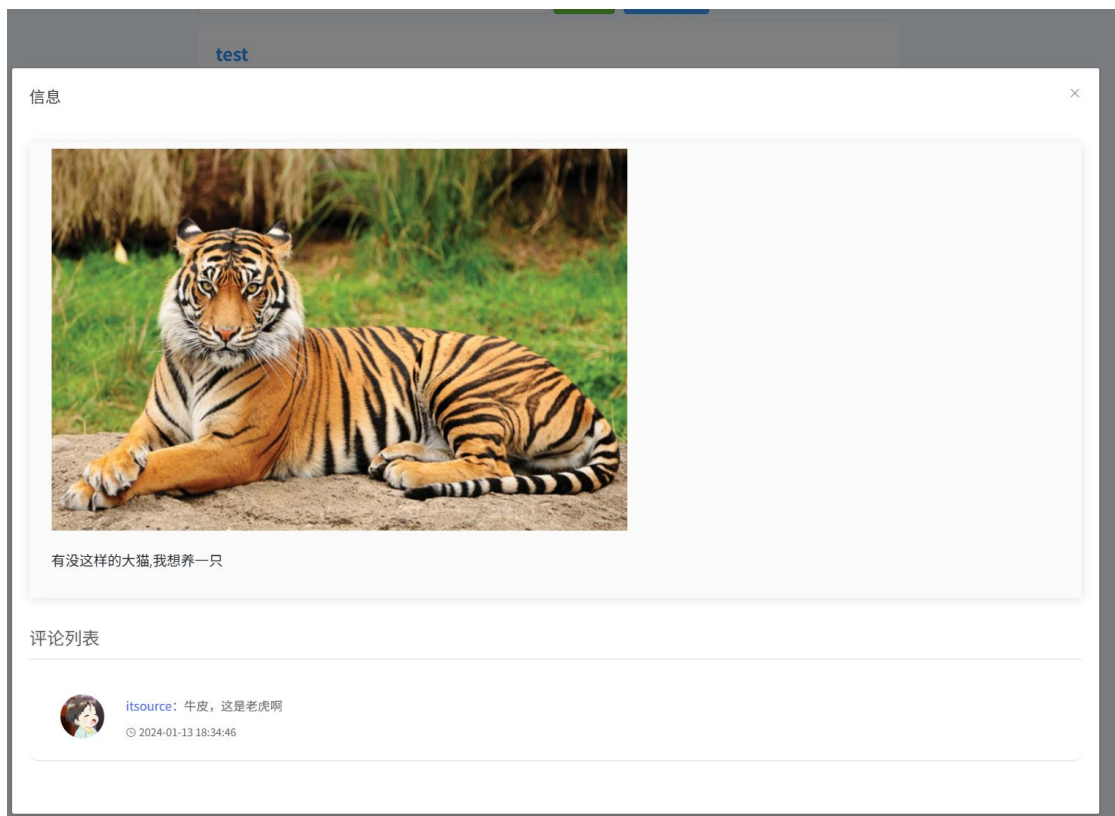


图 22 宠物活动评论界面

Fig.22 Pet activity comment screen

核心伪代码如下：

```
// 活动评论接口

public class CommentController {

    public Result addComment(@RequestBody Comment comment,
                             @RequestHeader("token") String token) {
        // 校验 token 有效性 → 存入数据库 → 返回结果
        return commentService.save(comment)
            ? Result.success("评论成功")
            : Result.error("发布失败");
    }

    @GetMapping("/list/{activityId}")
    public Result<List<Comment>> getComments(
        @PathVariable Integer activityId,
        @RequestParam int pageNum) { // 分页查询评论列表
        return Result.success(
            commentService.getCommentsByActivityId(activityId, pageNum));
    }
}
```

6 系统功能测试

系统测试是验证软件功能是否符合需求的重要环节。本系统按照功能模块划分为三个测试部分，每个部分均准备了详细的测试文档与测试用例。测试过程使用常规测试方法，重点关注基本功能实现与异常情况处理。

宠物建档功能测试用例如表 18 所示。

表 18 宠物建档功能测试功能测试

Table18 Pet Documentation Functional Testing			
用例名称	测试步骤	预期结果	测试结果
添加有效宠物信息	输入姓名 '小白'、类型 '狗'、年龄 8，点击保存	系统成功保存	成功
添加多个宠物信息	依次输入两只宠物信息 并保存	保存两条记录，提示成功	失败
缺少必填项	输入姓名'小白'，点击保存	提示'必填项缺失'，不保存	失败
年龄为负数	输入姓名'小白'、类型'狗'、年龄 '-1'，点击保存	提示'年龄不能为负数'，不保存	失败
姓名超长	输入超长姓名，点击保存	提示'姓名不超过 15 字符'，不保存	失败

宠物图片识别功能测试用例如表 19 所示

表 19 宠物图片识别功能测试功能测试

Table19 Pet picture recognition function test: Function test

用例名称	测试步骤	预期结果	测试结果
识别猫图片	上传并识别图片	结果: '猫'	成功
识别狗图片	上传并识别图片	结果: '狗'	失败
识别猫狗混合图	上传并识别图片	结果: '猫和狗'	失败
上传无效格式	上传 PDF 文件	提示'文件格式不支持'	失败
上传空文件	上传空文件并识别	提示'无法识别空文件'	失败
识别最大尺寸图片	上传并识别图片	成功识别, 速度可接受	失败

社区文章评论功能测试用例如表 20 所示

表 20 社区文章评论功能测试功能测试

Table20 Community article comments, feature testing, feature testing

用例名称	测试步骤	预期结果	测试结果
发布有效评论	输入内容点击发布	评论显示, 计数 + 1	成功
发布空评论	不输入内容点击发布	提示 ' 内容不能为空 ', 不发布	失败
发布超长评论	输入超字数点击发布	提示 ' 超出字数限制 ', 不发布	失败
含非法字符评论	输入非法字符点击发布	提示 ' 包含非法字符 ', 不发布	失败
字数边界测试	输入达到字符上限的内容, 点击发布	评论显示, 计数 + 1	失败

7 结论

本项目是一个基于深度学习的宠物领养平台, 目的是为铲屎官们以及爱猫爱狗人士提供一个信息共享的平台, 来帮助有领养寻宠或弃养需求的铲屎官快速找到心仪的宠物或者下家, 此外帮助那些无人认领的流浪宠物建立档案, 匹配合适的铲屎官。本设计通过改进卷积神经网络模型, 结合迁移学习与 Triplet Loss 函数, 可以有效提取宠物图像特征并且计算出相似度, 解决了同类宠物外形差异小、数据量不足的难题。平台的设计基于传统的前后端分离架构, 后端 Spring Boot 前端 Vue 结合稳定高效的 Mysql 数据库, 实现宠物建档、丢失找回、领养交流等基础业务。算法方面调用自训练的宠物识别模型接口, 实现宠物 1:1 和 1:N 的匹配识别, 不仅提升了领养流程的智能化水平, 还为救助机构提供了便捷的宠物信息管理工具。

本项目当前仍存在一些不足。一是当前宠物数据集收录的主要是国外的常见犬猫

品种,对本土品种以及一些特殊宠物的识别能力不足,后续会继续扩展训练的数据集,增加对本土宠物样本的学习训练,并且优化学习方法,提高学习效率和准确率。此外,现有匹配算法基本上都是依靠静态图像特征,没有深入分析宠物的动态行为,仅仅凭借一张照片来匹配宠物作为依据略显单薄,后续可以添加宠物视频或者动态图来辅助识别,进而构建多维适配模型。最后社区功能还可以进一步改进,目前的社区以基础评论和发帖为主,后面可集成个性化推荐算法,比如协同过滤算法,根据用户画像实现内容的精准推送。在技术架构层面,目前的模型推理服务依靠独立服务器,响应速度比较慢,后续可以将服务器上云,同时探索边缘计算方案以提升实时性。通过不断改进,可以构建一个集宠物社区、宠物领养、信息分享、宠物救助为一体的智慧宠物生态平台。

8 参考文献

- [1] 奚舒舒,李兰,张才宝.基于 SSD 网络的宠物狗检测与分类[J].信息技术与信息化,2019,(12):16-18.
- [2] 柯溢,卢亚玲.基于 VGG16 的宠物犬识别方法[J].电子制作,2020,(21):42-45.
- [3] 王稼祥.基于深度学习的宠物狗检测与识别研究[D].河南:河南师范大学,2021.
- [4] 汪洋,宋广佳,胡建华.一种基于 YOLO 的宠物图像识别方法[J].科技与创新,2022,(05):102-104.
- [5] 杨景,吴晓盼,甘恩硕.以深度学习为导向的智能宠物物联系统[J].福建电脑,2023,39(03):98-100.
- [6] 陈昭希,李如萍,李仪,等.基于深度学习的宠物照片拍摄指导微信小程序[J].信息技术与信息化,2023,(06):210-213.
- [7] 李宏铭.基于 YOLO 的宠物狗牵绳检测系统的设计与实现[J].电子制作,2023,31(13):49-52.
- [8] 李鸿鹏,方伟.犬鼻纹识别系统在警犬身份识别中的应用[J].当代畜牧,2023,(11):60-63.
- [9] 王起旺.基于深度感知的宠物个体检测与识别方法研究及应用[D].北京:北京邮电大学,2024.
- [10] 武占东.动物面部识别方法研究[D].吉林:吉林大学,2024.
- [11] 吴一.基于深度学习的宠物狗检测与分类[D].沈阳:沈阳工业大学,2024.
- [12] 李梦晗.基于深度学习的犬只分类与身份识别研究[D].山东:聊城大学,2024.
- [13] 吴嘉南.基于深度学习与自动液化的宠物表情操纵研究[D].四川:西南财经大学,2024.
- [14] 丁宇.基于深度学习的变电站设备识别与热故障诊断方法研究[D].广西大学,2024.
- [15] Kang K S ,Kim D ,Shin A S , et al.Accurate Automated Quantification of Dopamine Transporter PET Without MRI Using Deep Learning-based Spatial Normalization[J].Nuclear Medicine and Molecular Imaging,2024,58(6):354-363.
- [16] Rivière P A ,Vanderver D M ,Hagio T , et al.Automated Segmentation of Cardiac Inflammatory FDG PET Using a 3D-UNET Deep Learning Model[J].Journal of Nuclear Cardiology,2024,38(S):101978-

101978.

- [17] Lord S M ,Islamian P J ,Seyyedi N , et al.Deep-learning-based Attenuation Correction for 68Ga-DOTATATE Whole-body PET Imaging: A Dual-center Clinical Study.[J].Molecular imaging and radionuclide therapy,2024,33(3):138-146.
- [18] Bao N ,Zhang J ,Li Z , et al.CT-Less Whole-Body Bone Segmentation of PET Images Using a Multimodal Deep Learning Network.[J].IEEE journal of biomedical and health informatics,2024:36-88
- [19] Mozumder I A M ,Armand T P T ,Sumon I R , et al.Automated Pipeline for Robust Cat Activity Detection Based on Deep Learning and Wearable Sensor Data[J].Sensors,2024,24(23):7436-7436.
- [20] Arceo C E O ,Flores R U R ,Mancilla S C P .Design and Development of a Smart Pet Feeder with IoT and Deep Learning †[J].Engineering Proceedings,2024,82(1):63-63.
- [21] Salimi Y ,Mansouri Z ,Shiri I , et al.Deep Learning-Powered CT-Less Multitracer Organ Segmentation From PET Images: A Solution for Unreliable CT Segmentation in PET/CT Imaging.[J].Clinical nuclear medicine,2025:2365-2369

致 谢

毕设的顺利完成离不开众多老师同学支持帮助。衷心感谢指导老师[REDACTED]。从选题到系统的设计到最后的实现再到论文的修改，都离不开[REDACTED]的悉心指导。

感谢[REDACTED]各位老师，我在他们的课堂上打下了扎实的理论和专业基础。特别要感谢[REDACTED]，没有他天天鼓励我，我可能坚持不到最后。还有一路走来的同学们，我们互帮互助、互相出谋划策、相互激励，大大降低了延毕率。最后的最后，感谢[REDACTED]给了我这么好的学习环境和平台，我将继续努力，将所学知识应用于更多实际场景，不辜负学校的期望与培养。